From Local Improvement to Verified Repair: A Cross-Intervention Delivery Audit

Execution-grounded auditing and an exploratory preregistered prompt-only intervention

Abstract

Intermediate agent improvements need not become verified repairs. Across four historical intervention panels, selective edit supervision lowers validation negative log-likelihood by 31.5% while recorded repairs change from 12/24 to 11/24; feedback confirmation, attempted independent checking, and native-history exclusion provide no additional repairs. We audit 180 episodes, 2,451 tool events, and 21 tasks. Fixed-check execution on original, final-candidate, and reference source yields 314 qualified triples. Among failing candidates with zero-exit checks, all three states return zero in 16/16, 14/19, 16/21, and 36/36 units across panels: exit-status non-discrimination, not necessarily uninformative output. A locally preregistered prompt-only intervention adds 72 contemporary runs; SEAL achieves 6/24 repairs against baseline 5/24, with one successful flip and no demonstrated final-verification mechanism. Source-state replay and targeted cases distinguish execution, evidence interpretation, and artifact commitment. The contribution is a verifiable cross-intervention audit and a weak prospective test, not an established repair algorithm.

1. Introduction

Agent optimization often targets an intermediate quantity: demonstration likelihood, feedback use, test generation, or context cost. Repository repair, however, is judged on the delivered patch. A plausible operation must execute; a check must interrogate the relevant behavior; and the retained artifact must preserve the useful change. Better performance at one link need not improve the entire process.

Historical intervention outcomes motivate execution-grounded auditing; that audit then motivates a prospective prompt-only test. Neither retrospective reconstruction nor one successful flip isolates a prompt's causal mechanism. We combine paired interventions, mechanically reconstructed delivery events, three-state check discrimination, and bounded counterfactuals. We do not claim to discover ineffective self-tests, budget exhaustion, or patch regression first.



Figure 1. Repair-chain audit map. Links are descriptive; correct supplied candidates may bypass editing. This is not a causal decomposition.

2. Paired interventions and outcomes

The scaffold is Henao's Budget-Fit Single Agent V5 [1], deploying Gemma-4-31B QAT. Historical complete-agent panels generally allow eight minutes, 28 counted tools, 80 turns, and 8,192 output tokens. P8 instead refines supplied candidates with eight additional tools, twelve model requests, and 480 seconds. Historical seeds were uncontrolled; P120 fixes its seed. Panels are paired within protocol, not interchangeable randomized arms.

Panel and intervention	Intermediate result	Complete-agent outcome
P5: selective edit SFT	Validation NLL 0.2670 to 0.1828 (-31.5%)	BASE 12/24; LoRA 11/24
P6.2: feedback prompt	Exploration BASE 12/16; FB 13/16	Frozen confirmation: 13/24 each
P8: candidate checking	32 independent-check outputs; zero passed the format gate or executed	R/G/D: 4/16, 4/16, 3/16
P12: native-history exclusion / smaller thinking budget	Input tokens -35.9% / -27.4%	BASE/NH4096/NH1024: 7/12, 7/12, 6/12
P120: preregistered prompt-only SEAL	Contract/evidence/closure instructions; no weight update	A 5/24; B 5 PASS + 1 UNKNOWN; C 6/24

Table 1. Local objectives and repairs. Counts retain scheduled attempts, including P5's twelve UNKNOWN scores. P8 measures supplied-candidate preservation. NLL validation and feedback exploration are outside the 180-episode corpus. P120's 72 runs are separate; its original and adjudicated results appear in Section 6.

Selective supervision. A rank-16 adapter learns first source-edit calls from public OpenHands trajectories: 212 units/97 tasks for training, nineteen units/ten tasks for validation, partitioned by task and repository. NLL uses the same 6,888 target tokens, including formatting, paths, and edit context. Training is BF16-dequantized; deployment is QAT. Twelve development tasks with two repeats yield two favorable and three unfavorable flips. This does not refute SFT generally.

Feedback prompting. One paragraph distinguishes invalid probes from implementation defects and requests source-level responses to behavioral mismatches. Confirmation uses twelve other development tasks twice: two wins and two losses, with repeat-level advantage reversing. Equal counts do not establish equivalence.

Candidate checking. Eight frozen candidates, six incorrect and two correct, enter refinement-only R, generated-check G, and discriminator-oriented D arms twice each. All 32 G/D outputs fail the format gate; no independent check executes. This measures a failed interface, not usable verification. All eleven passes preserve correct inputs, ten without new source edits and one after restoration. No wrong input is repaired. Scheduling deviations constrain timing comparisons.

Native history. NH4096 excludes past thought content while retaining new reasoning; NH1024 also lowers thinking to 1,024 tokens. This is not learned summarization. Input savings accompany increased output tokens and no rescued BASE failures; NH1024 loses Pyflakes. Each arm has three timeouts. These development screens establish no cross-hardware efficiency gain.

3. From interventions to a mechanical delivery audit

E1 fixes P5/P6.2/P8/P12 at 48/48/48/36 episodes over 12/12/8/12 tasks respectively, totaling 21 distinct task identities. Repeats and panels share tasks. We report panels separately, without pooling them as independent episodes or attaching independence-based confidence intervals.

Actual source changes are reconstructed from diffs and tool feedback, including `write_file` and source-modifying `run_command`, not just `edit_file`. All 180 reconstructed final production-source states match archived delivery. This establishes source-state consistency under documented reconstruction assumptions, not exact runtime recovery. Semantic intent and Issue relevance do not determine the primary counts.

Our strict local condition requires an executed check with exit code zero **after the last actual production-source modification**. An earlier zero exit cannot validate a subsequently changed version. Within-command ordering is inspected. The condition means that such a check exists, not that the final check necessarily returned zero. We call these zero-exit local checks, never verified successful tests.

Panel	Actual new source change	Independent PASS / FAIL / UNKNOWN	FAIL among changed episodes	FAIL after last-change zero-exit
P5	28	23 / 13 / 12	5/28	4/25
P6.2	46	26 / 22 / 0	20/46	12/36
P8	10	11 / 37 / 0	9/10	1/2
P12	31	20 / 16 / 0	11/31	10/30

Table 2. Mechanical events and original independent outcomes. The last column conditions on actual source change, a subsequent qualifying zero-exit check, and a valid final score. P8's inherited-candidate preservation bypass is essential: no new edit need be appropriate for an already-correct candidate.

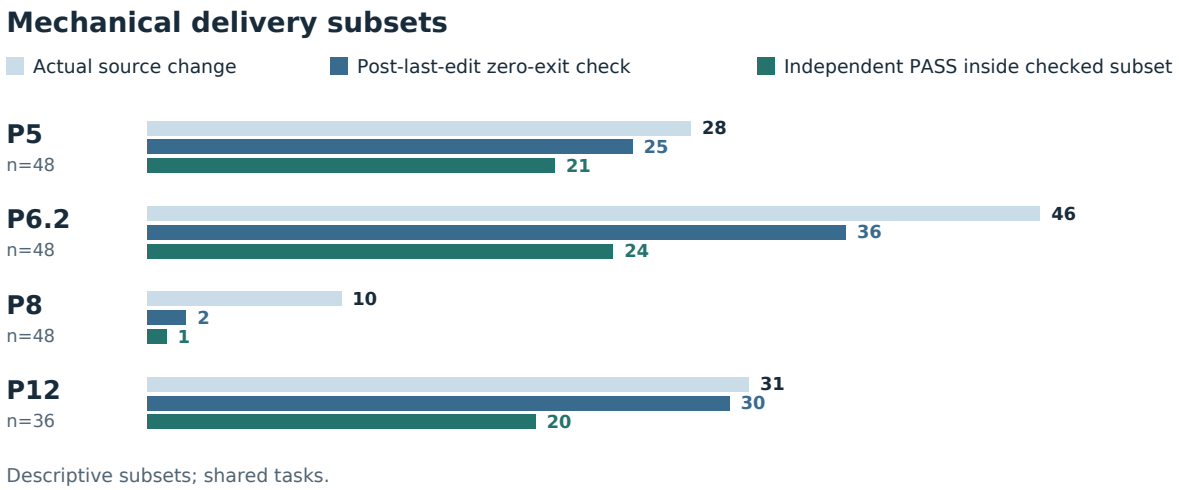


Figure 2. Nested descriptive subsets - source change, a qualifying later zero exit, and independent PASS - not necessary steps for every success. P8's one PASS inside the change branch restores an already-correct input. The broader any-after-change failure counts, 4/25, 14/38, 4/5, and 10/30, remain appendix diagnostics; especially P8's 4/5 must not replace the strict 1/2.

4. Three-state discrimination and temporal alignment

E4 asks whether an unchanged agent-written check discriminates **original buggy code, the actual final agent candidate, and reference repair**. Check body, command, auxiliary fixtures, and qualified environment are fixed across columns. Identity includes their hashes and the complete candidate patch. Production-source hashes before fixtures, after fixtures, and after checking prevent silently changing the candidate. Invalid application, missing inputs, or environment failures are UNKNOWN.

Eligibility is frozen before replay outcomes: 318 candidate-conditioned units yield 314 eligible triples, covering 408 historical events/130 episodes; four have incomplete or inconsistent fixtures. All 942 canonical executions qualify. Selection favors safely recoverable checks and is not representative of all checks.

Panel	Candidate exit=0 / failed-candidate units	All-three-zero / candidate-zero units	Failed episodes covered	Post-last-change candidate-zero / historical events
P5	16/20 (80.0%)	16/16 (100%)	8/13	9/9
P6.2	19/28 (67.9%)	14/19 (73.7%)	11/22	12/16
P8	21/29 (72.4%)	16/21 (76.2%)	23/37	0/1
P12	36/45 (80.0%)	36/36 (100%)	16/16	20/20

Table 3. Candidate-conditioned exit-status discrimination. Units deduplicate check/fixture/candidate/environment identities within panel. The all-three-zero column conditions further on candidate exit=0: 36/36 in P12 means all thirty-six zero-exit checks also return zero on original and reference, not thirty-six additional episodes. Reference nonzero outcomes do not remove candidates from the first denominator. Repeated tasks and checks remain dependent.

The three-way matrix sharpens the finding: in many failing-candidate units, process status does not distinguish any of the three source states. This does **not** mean the checks provide no information. Caught exceptions or printed failure messages may expose real differences without propagating a nonzero exit. Nor is every candidate zero exit an erroneous test: it can check a correctly implemented sub-behavior while another issue requirement fails. Conversely, four P12 units return zero on original and reference but nonzero on the actual candidate, demonstrating why the candidate column matters.

Table 3's last column separately restricts to the check's **historical position after the last source change**, with event rather than unit weighting. These are new-environment reexecutions, not original model-visible feedback. Small denominators, especially P8, limit comparison. P8 often preserves a supplied candidate without new source editing. Neither subset estimates historical agent misjudgment rates.

Event-weighted full-subset counts are 23/27, 26/36, 24/33, and 43/53; excluding fixture/scratch-byte conflicts gives 23/27, 26/34, 22/29, and 38/44. This is not complete historical-workspace recovery. New-environment independent labels change only P5 to 20/24 units; original UNKNOWN scores remain.

A separate historical alignment joins 549 checks to reconstructed production-source hashes: 213 differ from final source. Among 161 episodes with checks, fifteen last-check states differ from final source. Print-only checks, caught exceptions with zero exit, partial requirement coverage, and useful checks of old source are separate axes, not interchangeable failures. These conditional workspace identities do not establish historical loaded-module identity; old evidence is not automatically useless.

5. Bounded source-state and commitment cases

E2 scores reconstructed states on twenty qualified tasks: 166 new final scores agree in all 154 cases with valid comparable original scores; fourteen Selfies episodes remain unqualified. Of 153 intermediate-state attempts, 149 score validly. One P5 Billiard episode creates a passing intermediate patch and subsequently loses it. One P8 OpenAPI episode instead loses an initially supplied passing patch through a diagnostic revert. These are distinct existence cases, not evidence that regression explains most failures. Three captured self-checks from these two tasks return zero on both passing and failing states, though printed output can differ.

E3 replays two explicit budget-denied source edits unchanged. The official replacement helper rejects P6.2 Pyflakes because `old_string` is absent. P12 Pyflakes yields a workspace patch passing 25/25 tests. However, its chronology is read at step 30, successful `submit_patch` at step 31, then denied edit at step 32, approximately 34.21 seconds after submission. The submitted 238-byte patch contains only a reproduction script. The restored official runner returns an existing submitted snapshot, not automatically the later workspace; historical loaded-code identity is not retained. A denied edit would have produced a test-passing workspace patch, but successful final delivery was not demonstrated. Neither resubmission nor end-to-end counterfactual delivery was executed. One additional edit allowance is therefore not a demonstrated rescue.

6. Prospective exploratory SEAL intervention

SEAL (Specification-grounded Evidence and Artifact Locking) is prompt-only: A is exact BF_V5; B adds Stop & Seal budget/closure guidance; C adds Issue-grounded contract and final-source evidence instructions to B. Only `system.md` differs; weights, tools, and scaffold are unchanged. Task identities, prompts, seed 20261018, contrasts, and UNKNOWN handling were locally preregistered in a hash-pinned lock before main execution, not registered publicly. Six exposed engineering smokes are excluded; prompts were not selected using their scores.

Twenty-four tasks from 24 repositories receive one A/B/C run each: **72 actual contemporary generations**, without historical baseline reuse. A task's arms share the physical GPU/profile; six arm-order permutations occur four times each. Limits are 480 seconds, 28 charged tools, 80 turns, 8,192 output tokens and 4,096 thinking tokens; temperature/top-p are 0.2/0.95. The panel has 22 reused DEV tasks and two TRAIN-origin candidates without project-model exposure found in a bounded scan, not 24 fresh heldout tasks.

Original outcomes are A: 5 PASS/19 FAIL; B: 5 PASS/18 FAIL/1 UNKNOWN; C: 6 PASS/18 FAIL. C-A has one win, no losses; its unadjusted exact discordant diagnostic is p=1.0. Original C-B registered net bounds are 0-1 repair. All arms fail both project-unexposed candidates; the sole flip, Abjad, is reused DEV. These results establish no stable gain.

B's Marshmallow collection error remains UNKNOWN in the primary record. Separate CPU diagnosis attributes it to a candidate-added call passing `cls` twice to a bound metaclass method. Original-source and override-removal controls remove that exception; the latter is not a repair. A separate assistant adjudication classifies the candidate FAIL: A/B/C then have 5/5/6 repairs, still 71 strict-valid raw scores plus one source-attributed failure, not 72 valid scoring receipts.

In Abjad, C executes a repository example for the second reported behavior, interprets its output, then changes a constructor default and replaces mutation-during-iteration with filtering. A/B also discuss that behavior, but A implements only filtering and B retains debugging. A/B fail two default-value assertions; C passes eight tests. Post-hoc CPU ablations use the unchanged frozen scorer: default-only passes 8/8; filtering-only passes 6/8 and fails the same two constructor-default assertions. The default change alone reproduces C's test success. This isolates code contributions under existing tests, not prompt causality.

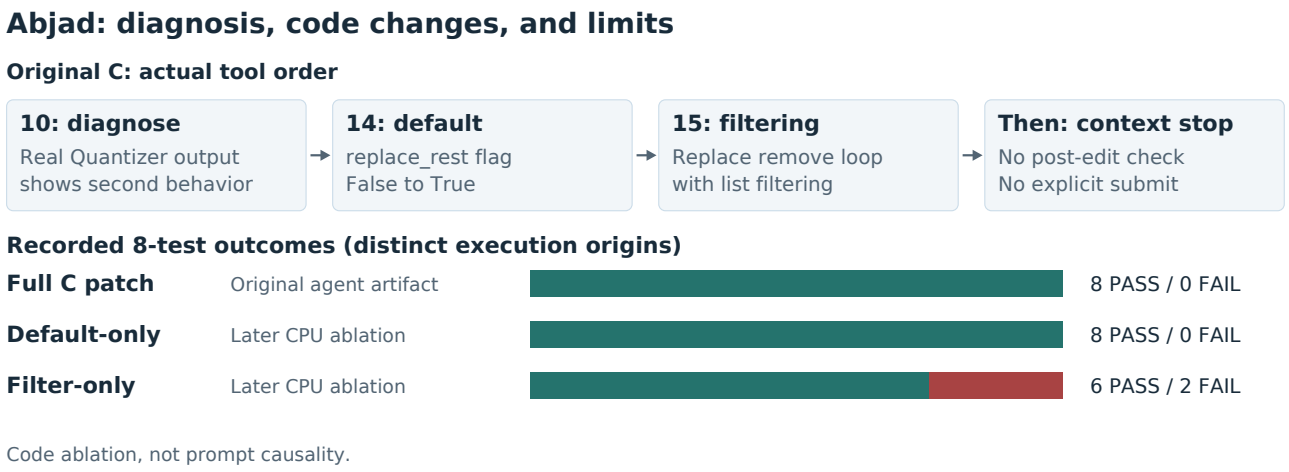


Figure 3. Abjad evidence and patch ablation. C's original agent artifact and the two later CPU ablations have distinct provenance. Only the default-value change is needed for the existing eight-test PASS; this does not identify prompt causality or broader correctness. C never performed post-edit verification or explicitly submitted.

No failing check occurs between C's two edits. C performs no post-edit check; all three arms exceed context and never explicitly submit. Its PASS is the returned workspace patch, not demonstrated final-verification or commitment success. This is an exploratory diagnosis-to-action case, with unfulfilled SEAL mechanisms.

7. Related work and scope of novelty

SWE-bench supplies execution-based repository-repair evaluation [2]. SWE-Gym trains agents and trajectory-based verifiers [3]; SWE-Prime selects trajectories and loss-bearing segments while preserving context [4]. They establish effective learning approaches, whereas our selective-edit result separates improved conditional target fit from complete-agent outcomes; it neither reproduces nor refutes their methods.

SWE-Replay reuses trajectory prefixes and branches to improve test-time scaling [5]. Our replay independently scores archived source states; it does not sample policy continuations or estimate recovery probability. ExecCritic separates and trains test construction and repair, qualifying and freezing generated tests [6]. Our fixed-check experiment audits discrimination retrospectively; P8's failed format gate provides no comparison with usable independent-test feedback.

Trajectory research already studies successful versus failed strategies [7], temporal onset and recovery of failures [8], and divergence between submission and test-verified resolution, including edits to already-correct code [9]. Trial by Trace separately evaluates harness reliability and task success [10]. Our contribution links intervention outcomes to verifiable delivery, candidate-conditioned temporal check matrices, and a weak prospective prompt-only test. We claim neither the first failure taxonomy nor causal attribution to individual harness or prompt components.

8. Limitations, artifacts, and conclusion

The panels are small, development-directed, reused, heterogeneous, and not an independent generalization study. They establish no equivalence, hidden-score gain, or general inefficacy of fine-tuning or verification. No cross-hardware timing is pooled. Counterfactual replay depends on reconstructed inputs and qualified new environments; task dependence and eligibility selection limit extrapolation. Fixed scripts and auxiliary files do not control probe-internal randomness, such as generated random bytes.

Semantic labels are secondary: 148 targeted assistant-inspected units include 78 intent packets, 61 checks, and nine probes. Check judgments are 43 limited-relevance, eight non-covering, ten uncertain. Asgineer extremes do not establish 90% boundary coverage. Four core historical cases receive additional claim-level assistant confirmation; P120's Abjad and Marshmallow reviews remain targeted, not semantic review of all 72 runs. Human confirmations remain zero; no assistant review is called human certification.

The public artifact provides anonymized statistics, source hashes, Abjad patches, and CPU verification scripts. It excludes private tests, the task scorer, full snapshots, and raw trajectories. Statistical recomputation is not new agent execution; reproducing private test outcomes still requires unavailable scoring assets.

The main contribution is identifying auditable breaks between local objectives and verified repair. SEAL is an exploratory intervention; lower loss, zero-exit checks, or a passing uncommitted workspace alone do not establish improved complete-task delivery.

References

1. Henao. *Gemma 4 Agent: Budget-Fit Single Agent, Version 5*. 2026.
2. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*. 2023.
3. Pan et al. *Training Software Engineering Agents and Verifiers with SWE-Gym*. ICML, 2025.
4. Zheng et al. *SWE-Prime: Fewer Trajectories, Better Performance*. 2026.
5. Ding and Zhang. *SWE-Replay: Efficient Test-Time Scaling for Software Engineering Agents*. 2026.
6. Tao et al. *ExecCritic: Learn to Test, Test to Improve for Coding Agents*. 2026.
7. Majgaonkar et al. *Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories*. 2025.
8. Zhao et al. *Failure as a Process: An Anatomy of CLI Coding Agent Trajectories*. 2026.
9. Mehta. *Confident and Wrong: Silent Semantic Failures in Coding Agents*. 2026.
10. Cao et al. *Trial by Trace: Benchmarking AI Agent Harnesses for Reliability and Failure Attribution*. 2026.